```
separate (IDL.SEM_PHASE)¶    --|---------------------------------------------------------------------------------------¶    --| INSTANT¶    --|
----------------------------------------------------------------------------------------¶package body INSTANT is¶  use DEF_UTIL;¶  use VIS_UTIL;
¶  use MAKE_NOD;¶  use GEN_SUBS;¶  use HOM_UNIT;¶  use EXP_TYPE, EXPRESO;¶  use REQ_UTIL;¶  use SET_UTIL;¶  use NOD_WALK;¶¶  type FORMAL_ARRAY_DATA is
 record¶    ID¬ : TREE;¶    SYM¬ : TREE;¶    ACTUAL : TREE;¶  end record;¶¶  type FORMAL_ARRAY_TYPE is array (Positive range¬<>) of FORMAL_ARRAY_DATA;
¶¶  procedure RESOLVE_GENERIC_FORMALS (NODE_HASH : in out NODE_HASH_TYPE; GENERIC_PARAM_S :¬TREE; GENERAL_ASSOC_S : TREE;¬NEW_DECL_S : out TREE; H : H
_TYPE);¶¶  function COUNT_GENERIC_FORMALS (ITEM_S : TREE) return Natural;¶¶  procedure SPREAD_GENERIC_FORMALS (ITEM_S : TREE; FORMAL :¬out FORMAL_ARRA
Y_TYPE);¶¶  procedure WALK_GENERIC_ACTUAL (NODE_HASH : in out NODE_HASH_TYPE; FORMAL_ID :¬TREE; ACTUAL_EXP : in out TREE; H : H_TYPE);¶¶  procedure CO
NSTRUCT_INSTANCE_DECL (NODE_HASH : in out NODE_HASH_TYPE; FORMAL_ID : TREE;¬ACTUAL_EXP : TREE; NEW_DECL_LIST : in out SEQ_TYPE; H : H_TYPE);¶¶  proced
ure FIX_DECLS_AND_SUBSTITUTE (DECL_S : TREE; NODE_HASH : in out NODE_HASH_TYPE; H¬: H_TYPE);¶¶      --|-------------------------------------------
--------------------------------------------¶      --|¶        -- $$$$ SHOULDN'T BE HERE¶  function LENGTH (L : SEQ_TYPE) return¬Natural is¶    TEMP¬:
 SEQ_TYPE := L;¶    COUNT¬: Natural¬ := 0;¶  begin¶    while¬not IS_EMPTY (TEMP)¬loop¶      COUNT := COUNT + 1;¶      TEMP  := TAIL¬(TEMP);¶    end lo
op;¶    return COUNT;¶  end LENGTH;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  p
rocedure WALK_INSTANTIATION (UNIT_ID¬: TREE; INSTANTIATION : TREE;¬H : H_TYPE) is¶    GEN_ASSOC_S : constant TREE := D (AS_GENERAL_ASSOC_S, INSTANTIAT
ION);¶    NAME¬       : TREE¬  := D (AS_NAME, INSTANTIATION);¶¶    UNIT_DEF   : constant TREE := GET_DEF_FOR_ID (UNIT_ID);¶    GENERIC_ID : TREE;¶    N
ODE_HASH  : NODE_HASH_TYPE;¶    NEW_DECL_S : TREE;¶    UNIT_SPEC  : TREE;¶  begin¶¶¬      -- RESOLVE THE GENERIC UNIT NAME¶    NAME := WALK_NAME (DN_G
ENERIC_ID, NAME);¶    D (AS_NAME, INSTANTIATION, NAME);¶    GENERIC_ID := GET_NAME_DEFN (NAME);¶¶¬      -- QUIT IF NO¬GENERIC UNIT WAS FOUND¶    if GE
NERIC_ID =¬TREE_VOID¬then¶      return;¶    end if;¶¶¬      -- SUBSTITUTE¬INSTANCE NAME FOR GENERIC NAME¶    INSERT_NODE_HASH (NODE_HASH, UNIT_ID, GEN
ERIC_ID);¶¶¬      -- WITHIN THE¬NEW REGION¶    declare¶      H :¬H_TYPE :=¬WALK_INSTANTIATION.H;¶      S :¬S_TYPE;¶    begin¶      ENTER_REGION (UNIT_
DEF,¬H, S);¶      H.IS_IN_SPEC := False;¶¶¬¬    -- BUT REGION NAME NOT VISIBLE AS ENCLOSING REGION WHILE¶¬¬    -- ... RESOLVING FORMALS¶      DI (XD_L
EX_LEVEL, UNIT_DEF, 0);¶¶¬¬     -- RESOLVE FORMAL PARAMETERS¶      RESOLVE_GENERIC_FORMALS¬(NODE_HASH, D (SM_GENERIC_PARAM_S, GENERIC_ID), GEN_ASSOC_S,
 NEW_DECL_S, H);¶      D (SM_DECL_S,¬INSTANTIATION, NEW_DECL_S);¶¶¬¬     -- CONSTRUCT NEW UNIT SPEC¶      UNIT_SPEC := D (SM_SPEC, GENERIC_ID);¶      i
f UNIT_SPEC.TY = DN_PACKAGE_SPEC¬then¶        declare¶¬DECL_S1 :¬TREE := D¬(AS_DECL_S1, UNIT_SPEC);¶¬DECL_S2 :¬TREE := D¬(AS_DECL_S2, UNIT_SPEC);¶
    begin¶¬¬¬¬¬-- RESTORE VISIBILITY OF NEW UNIT¶¬DI (XD_LEX_LEVEL, UNIT_DEF, H.LEX_LEVEL);¶¬MAKE_DEF_VISIBLE (UNIT_DEF);¶¶¬H.IS_IN_SPEC := True;¶¬FIX_
DECLS_AND_SUBSTITUTE (DECL_S1, NODE_HASH, H);¶¬H.IS_IN_SPEC := False;¶¬FIX_DECLS_AND_SUBSTITUTE (DECL_S2, NODE_HASH, H);¶        end;¶      elsif UNIT
_SPEC.TY = DN_TASK_SPEC¬then¶        H.IS_IN_SPEC := True;¶      end¬if;¶      SUBSTITUTE (UNIT_SPEC, NODE_HASH,¬H);¶      D (SM_SPEC, UNIT_ID, UNIT_S
PEC);¶¶      LEAVE_REGION (UNIT_DEF,¬S);¶    end;¶  end WALK_INSTANTIATION;¶      --|----------------------------------------------------------------
--------------------------¶      --|¶  procedure RESOLVE_GENERIC_FORMALS (NODE_HASH : in out NODE_HASH_TYPE; GENERIC_PARAM_S :¬TREE; GENERAL_ASSOC_S :
 TREE;¬NEW_DECL_S : out TREE; H : H_TYPE) is¶¶    FORMAL_COUNT : constant Natural := COUNT_GENERIC_FORMALS (GENERIC_PARAM_S);¶¶    ACTUAL_LIST¬: SEQ_T
YPE := LIST (GENERAL_ASSOC_S);¶    ACTUAL¬: TREE;¶    ACTUAL_SYM¬: TREE;¶    ACTUAL_EXP¬: TREE;¶    NEW_ACTUAL_LIST¬: SEQ_TYPE := (TREE_NIL, TREE_NIL)
;¶¶    ACTUAL_SUB¬: Natural¬ := 0;¶    FORMAL_SUB¬: Natural¬ := 0;¶    FIRST_NAMED_SUB¬: Natural¬ := 0;¶    NEW_DECL_LIST¬: SEQ_TYPE := (TREE_NIL, TRE
E_NIL);¶    UNIT_DESC¬: TREE;¶¶    FORMAL¬ : FORMAL_ARRAY_TYPE (1 .. FORMAL_COUNT);¶    ACTUAL_TO_FORMAL : array (1 .. LENGTH (ACTUAL_LIST)) of¬Natura
l := (others => 0);¶  begin¶¶¬      -- SPREAD THE¬FORMAL PARAMETERS¶    SPREAD_GENERIC_FORMALS (GENERIC_PARAM_S, FORMAL);¶¶¬      -- FOR EACH POSITION
AL ACTUAL¶    while¬not IS_EMPTY (ACTUAL_LIST) and then HEAD (ACTUAL_LIST).TY /= DN_ASSOC¬loop¶      POP¬(ACTUAL_LIST, ACTUAL);¶¶¬¬     -- IF¬THERE ARE
¬TOO MANY POSITIONALS¶      ACTUAL_SUB :=¬ACTUAL_SUB + 1;¶      if ACTUAL_SUB¬> FORMAL'LAST then¶¶¬¬¬  -- PUT OUT ERROR¶        ERROR (D (LX_SRCPOS, A
CTUAL), "TOO MANY ACTUALS");¶¶¬¬¬  -- ELSE¶      else¶¶¬¬¬  -- SAVE¬ACTUAL¶        ACTUAL_TO_FORMAL (ACTUAL_SUB) := ACTUAL_SUB;¶        FORMAL (ACTUAL
_SUB).ACTUAL    := ACTUAL;¶      end¬if;¶    end loop;¶    FIRST_NAMED_SUB¬:= ACTUAL_SUB + 1;¶¶¬      -- FOR EACH ACTUAL FOLLOWING THE POSITIONALS¶
 while¬not IS_EMPTY (ACTUAL_LIST) loop¶      POP¬(ACTUAL_LIST, ACTUAL);¶      ACTUAL_SUB :=¬ACTUAL_SUB + 1;¶¶¬¬    -- CHECK THAT IT IS NAMED¬AND GET S
YMBOL¶      if ACTUAL.TY /= DN_ASSOC then¶        ERROR (D (LX_SRCPOS, ACTUAL), "POSITIONAL PARAMETER AFTER NAMED");¶      else¶        ACTUAL_SYM :=
D (LX_SYMREP, D (AS_USED_NAME, ACTUAL));¶¶¬¬¬  -- SEARCH FORMALS¬FOR (UNIQUE) MATCHING ID¶        FORMAL_SUB := 0;¶        for I in FIRST_NAMED_SUB ..
 FORMAL'LAST loop¶¬if ACTUAL_SYM = FORMAL (I).SYM then¶¬  if FORMAL_SUB = 0¬then¶¬    FORMAL_SUB := I;¶¬  else¶¬    ERROR¬(D (LX_SRCPOS, ACTUAL), "AMB
IGUOUS GENERIC ARGUMENT ASSOC");¶¬    FORMAL_SUB := 0;¶¬    exit;¶¬  end if;¶¬end if;¶        end loop;¶¶¬¬¬  -- IF OK MATCH, SAVE ACTUAL¶        if F
ORMAL_SUB = 0 then¶¬ERROR (D (LX_SRCPOS, ACTUAL),¬"NO MATCHING GENERIC FORMAL");¶        else¶¬ACTUAL_TO_FORMAL (ACTUAL_SUB)¬:= FORMAL_SUB;¶¬FORMAL (F
ORMAL_SUB).ACTUAL¬:= ACTUAL;¶        end if;¶      end¬if;¶    end loop;¶¶¬      -- RESOLVE THE ACTUAL PARAMETERS¶¬      -- FOR EACH FORMAL¶    for I¬
in FORMAL'RANGE loop¶      ACTUAL := FORMAL (I).ACTUAL;¶¶¬¬     -- IF¬AN ACTUAL¬WAS EXPLICITLY GIVEN¶      if ACTUAL /= TREE_VOID then¶¶¬¬¬  -- STRIP N
AME FROM ARGUMENT¬ASSOCIATION¶        if ACTUAL.TY = DN_ASSOC then¶¬ACTUAL_EXP := D (AS_EXP, ACTUAL);¶¬¬¬¬¬-- FIXUP USED_NAME_ID¶¬declare¶¬  USED_NAME
: TREE := D (AS_USED_NAME, ACTUAL);¶¬begin¶¬  if USED_NAME.TY =¬DN_USED_OBJECT_ID then¶¬    D (AS_USED_NAME, ACTUAL, MAKE_USED_NAME_ID_FROM_OBJECT (US
ED_NAME));¶¬  elsif USED_NAME.TY = DN_STRING_LITERAL then¶¬    D (AS_USED_NAME, ACTUAL, MAKE_USED_OP_FROM_STRING (USED_NAME));¶¬  end if;¶¬end;¶¬D (SM
_DEFN, D (AS_USED_NAME, ACTUAL), TREE_VOID);¶        else¶¬ACTUAL_EXP := ACTUAL;¶        end if;¶¶¬¬¬  -- AND RESOLVE THE ACTUAL¶        if ACTUAL_EXP
.TY = DN_STRING_LITERAL and then FORMAL (I).ID.TY in CLASS_SUBPROG_NAME then¶¬ACTUAL_EXP := MAKE_USED_OP_FROM_STRING (ACTUAL_EXP);¶        end if;¶
    WALK_GENERIC_ACTUAL (NODE_HASH,¬FORMAL (I).ID, ACTUAL_EXP, H);¶¶¬¬¬  -- ELSE¬-- SINCE NO ACTUAL WAS GIVEN¶      else¶¶¬¬¬  -- IN CASE NO DEFAULT,

 USE VOID ACTUAL_EXP¶        ACTUAL_EXP := TREE_VOID;¶¶¬¬¬  -- IF PARAMETER IS OBJECT WITH DEFAULT¶        if FORMAL (I).ID.TY =¬DN_IN_ID and then D¬(
SM_INIT_EXP, FORMAL (I).ID) /= TREE_VOID then¶¶¬¬¬¬¬-- USE THE DEFAULT¶¬ACTUAL_EXP := D (SM_INIT_EXP,¬FORMAL (I).ID);¶¬SUBSTITUTE (ACTUAL_EXP, NODE_HAS
H, H);¶¶¬¬¬¬¬-- ELSE IF PARAMETER IS SUBPROGRAM WITH¬DEFAULT¶        elsif FORMAL (I).ID.TY in CLASS_SUBPROG_NAME and then D (SM_UNIT_DESC, FORMAL (I).
ID).TY /=¬DN_NO_DEFAULT then¶¶¬¬¬¬¬-- IF IT IS A NAME DEFAULT¶¬UNIT_DESC¬:= D (SM_UNIT_DESC,¬FORMAL (I).ID);¶¬if UNIT_DESC.TY = DN_NAME_DEFAULT then¶¶¬
¬¬        -- USE THE (ALREADY RESOLVED) NAME¶¬  ACTUAL_EXP := D (AS_NAME, UNIT_DESC);¶¬  SUBSTITUTE (ACTUAL_EXP, NODE_HASH, H);¶¶¬¬¬¬        -- ELSE
-- SINCE IT IS A BOX DEFAULT¶¬else¶¶¬¬¬¬        -- CONSTRUCT NAME TO RESOLVE¶¬  if FORMAL (I).ID.TY = DN_OPERATOR_ID then¶¬    ACTUAL_EXP := MAKE_USED
_OP (LX_SYMREP => D (LX_SYMREP, FORMAL (I).ID), LX_SRCPOS => D¬(LX_SRCPOS, GENERAL_ASSOC_S));¶¬  else¶¬    ACTUAL_EXP := MAKE_USED_OBJECT_ID (LX_SYMRE
P => D (LX_SYMREP, FORMAL (I).ID), LX_SRCPOS => D (LX_SRCPOS, GENERAL_ASSOC_S));¶¬  end if;¶¶¬¬¬¬        -- AND RESOLVE IT¶¬  WALK_GENERIC_ACTUAL (NOD
E_HASH, FORMAL (I).ID, ACTUAL_EXP, H);¶¬end if;¶¶¬¬¬¬¬-- ELSE -- SINCE NO¬DEFAULT GIVEN¶        else¶¬ERROR (D (LX_SRCPOS, GENERAL_ASSOC_S), "NO VALUE
GIVEN FOR GENERIC PARAMETER - " & PRINT_NAME (FORMAL (I).SYM));¶        end if;¶      end¬if;¶¶¬¬     -- CONSTRUCT DECLARATION FOR GENERIC ACTUAL¶
 CONSTRUCT_INSTANCE_DECL¬(NODE_HASH, FORMAL (I).ID, ACTUAL_EXP, NEW_DECL_LIST, H);¶¶¬¬     -- AND UPDATE ACTUAL¶      ACTUAL := FORMAL (I).ACTUAL;¶
  if ACTUAL.TY = DN_ASSOC¬then¶        D¬(AS_EXP, ACTUAL, ACTUAL_EXP);¶      else¶        FORMAL (I).ACTUAL := ACTUAL_EXP;¶      end¬if;¶    end loop;
¶¶¬      -- CONSTRUCT AND SAVE LIST OF RESOLVED ACTUALS¶    for I¬in ACTUAL_TO_FORMAL'RANGE loop¶      if ACTUAL_TO_FORMAL (I)¬/= 0 then¶        NEW_A
CTUAL_LIST := APPEND (NEW_ACTUAL_LIST, FORMAL (ACTUAL_TO_FORMAL (I)).ACTUAL);¶      end¬if;¶    end loop;¶    LIST (GENERAL_ASSOC_S, NEW_ACTUAL_LIST);
¶¶    NEW_DECL_S := MAKE_DECL_S¬(LIST => NEW_DECL_LIST);¶  end RESOLVE_GENERIC_FORMALS;¶      --|--------------------------------------------------------
-----------------------------------¶      --|¶  function COUNT_GENERIC_FORMALS (ITEM_S : TREE) return Natural is¶    ITEM_LIST  : SEQ_TYPE := LIST
(ITEM_S);¶    ITEM¬      : TREE;¶    ITEM_KIND  : NODE_NAME;¶    ITEM_COUNT : Natural  := 0;¶  begin¶¶¬      -- FOR EACH ELEMENT OF GENERIC FORMAL DECL
ARATION LIST¶    while¬not IS_EMPTY (ITEM_LIST) loop¶      POP¬(ITEM_LIST, ITEM);¶¶¬¬     -- IF¬IT IS AN IN OR AN IN OUT DECLARATION¶      ITEM_KIND :=
 ITEM.TY;¶      if ITEM_KIND = DN_IN or¬ITEM_KIND¬= DN_IN_OUT then¶¶¬¬¬  -- ADD THE NUMBER¬OF DECLARED IDENTIFIERS¶        ITEM_COUNT := ITEM_COUNT +
LENGTH (LIST (D (AS_SOURCE_NAME_S,¬ITEM)));¶¶¬¬¬  -- ELSE¬IF IT IS ANYTHING ELSE OTHER THAN A PRAGMA¶      elsif ITEM_KIND /= DN_PRAGMA then¶¶¬¬¬  --
ADD ONE DECLARATION¶        ITEM_COUNT := ITEM_COUNT + 1;¶      end¬if;¶    end loop;¶¶¬      -- RETURN THE¬COUNT¶    return ITEM_COUNT;¶  end COUNT_G
ENERIC_FORMALS;¶      --|-------------------------------------------------------------------------------------------¶      --|¶  procedure SPREAD_GENE
RIC_FORMALS (ITEM_S : TREE; FORMAL :¬out FORMAL_ARRAY_TYPE) is¶    ITEM_LIST  : SEQ_TYPE := LIST (ITEM_S);¶    ITEM¬      : TREE;¶    ITEM_KIND  : NODE
_NAME;¶    ID_LIST    : SEQ_TYPE;¶    ID¬        : TREE;¶    ITEM_COUNT : Natural  := 0;¶¶  begin¶¶¬      -- FOR EACH ELEMENT OF GENERIC FORMAL DECLARATI
ON LIST¶    while¬not IS_EMPTY (ITEM_LIST) loop¶      POP¬(ITEM_LIST, ITEM);¶¶¬¬     -- IF¬IT IS AN IN OR AN IN-OUT DECLARATION¶      ITEM_KIND := ITEM
.TY;¶      if ITEM_KIND = DN_IN or¬ITEM_KIND¬= DN_IN_OUT then¶¶¬¬¬  -- FOR EACH DECLARED IDENTIFIER¶        ID_LIST := LIST (D (AS_SOURCE_NAME_S, ITEM
));¶        while not IS_EMPTY (ID_LIST) loop¶¬POP (ID_LIST, ID);¶¶¬¬¬¬¬-- FILL IN DATA FOR¬IN OR IN-OUT PARAMETER¶¬ITEM_COUNT¬        :=¬ITEM_COUNT + 1
;¶¬FORMAL (ITEM_COUNT).ID      :=¬ID;¶¬FORMAL (ITEM_COUNT).SYM     :=¬D (LX_SYMREP, ID);¶¬FORMAL (ITEM_COUNT).ACTUAL :=¬TREE_VOID;¶        end loop;¶¶¬¬
¬  -- ELSE¬IF IT IS ANYTHING ELSE OTHER THAN A PRAGMA¶      elsif ITEM_KIND /= DN_PRAGMA then¶¶¬¬¬  -- FILL¬IN DATA FOR FORMAL TYPE OR FORMAL SUBPROGR
AM¶        ITEM_COUNT¬¬      := ITEM_COUNT + 1;¶        ID¬¬      := D¬(AS_SOURCE_NAME, ITEM);¶        FORMAL (ITEM_COUNT).ID       := ID;¶        FORMAL
(ITEM_COUNT).SYM    := D¬(LX_SYMREP, ID);¶        FORMAL (ITEM_COUNT).ACTUAL := TREE_VOID;¶      end¬if;¶    end loop;¶  end SPREAD_GENERIC_FORMALS;¶
    --|--------------------------------------------------------------------------------------¶      --|¶  procedure WALK_GENERIC_ACTUAL (NODE_HA
SH : in out NODE_HASH_TYPE; FORMAL_ID :¬TREE; ACTUAL_EXP : in out TREE; H : H_TYPE) is¶    BASE_TYPE : TREE;¶    TYPESET    : TYPESET_TYPE;¶    HEADER
   : TREE;¶  begin¶    case FORMAL_ID.TY is¶      when DN_IN_ID¬| DN_IN_OUT_ID =>¶        BASE_TYPE := GET_BASE_TYPE (FORMAL_ID);¶        SUBSTITUTE (
BASE_TYPE, NODE_HASH, H);¶        EVAL_EXP_TYPES (ACTUAL_EXP, TYPESET);¶        REQUIRE_TYPE (GET_BASE_TYPE (BASE_TYPE), ACTUAL_EXP, TYPESET);¶
 ACTUAL_EXP := RESOLVE_EXP (ACTUAL_EXP, TYPESET);¶¶      when DN_TYPE_ID =>¶        ACTUAL_EXP := WALK_TYPE_MARK (ACTUAL_EXP);¶¬¬    -- $$$$ NEED TO¬C
HECK COMPATIBILITY¶¶      when DN_PRIVATE_TYPE_ID¬| DN_L_PRIVATE_TYPE_ID =>¶        ACTUAL_EXP := WALK_TYPE_MARK (ACTUAL_EXP);¶¬¬    -- $$$$ NEED TO¬C
HECK COMPATIBILITY¬FOR PRIVATE¶¶      when CLASS_SUBPROG_NAME¬=>¶        HEADER := D¬(SM_SPEC,¬FORMAL_ID);¶        SUBSTITUTE (HEADER, NODE_HASH, H);¶
        ACTUAL_EXP := WALK_HOMOGRAPH_UNIT (ACTUAL_EXP, HEADER);¶¶      when others =>¶        Put_Line ("!! BAD GENERIC ACTUAL ID");¶        raise Pro
gram_Error;¶    end case;¶  end WALK_GENERIC_ACTUAL;¶      --|-----------------------------------------------------------------------------------
---¶      --|¶  procedure CONSTRUCT_INSTANCE_DECL (NODE_HASH : in out NODE_HASH_TYPE; FORMAL_ID : TREE;¬ACTUAL_EXP : TREE; NEW_DECL_LIST : in out SEQ_
TYPE; H : H_TYPE) is¶    SYMREP       : TREE := D (LX_SYMREP, FORMAL_ID);¶    SRCPOS       : TREE;¶    NEW_ID       : TREE;¶    NEW_DEF      : TREE;¶
   NEW_DECL     : TREE;¶    DEFN¬        : TREE;¶    SUBTYPE_NODE : TREE;¶    HEADER       : TREE;¶  begin¶    if ACTUAL_EXP /= TREE_VOID then¶      SR
CPOS := D (LX_SRCPOS,¬ACTUAL_EXP);¶    else¶      SRCPOS := TREE_VOID;¶    end if;¶    if SYMREP.TY = DN_TXTREP then¶      SYMREP := STORE_SYM (PRINT_
NAME (SYMREP));¶      D (LX_SYMREP,¬FORMAL_ID, SYMREP);¶    end if;¶¶    case FORMAL_ID.TY is¶      when DN_IN_ID¬=>¶        SUBTYPE_NODE := D (SM_OBJ
_TYPE,¬FORMAL_ID);¶        SUBSTITUTE (SUBTYPE_NODE, NODE_HASH, H);¶        NEW_ID := MAKE_CONSTANT_ID (SM_OBJ_TYPE => SUBTYPE_NODE, SM_INIT_EXP =>¬AC
TUAL_EXP);¶        D¬(SM_FIRST, NEW_ID, NEW_ID);¶        NEW_DECL :=¬MAKE_CONSTANT_DECL (LX_SRCPOS¬=> SRCPOS, AS_SOURCE_NAME_S => MAKE_SOURCE_NAME_S¬(
LX_SRCPOS => SRCPOS, LIST =>¬SINGLETON¬(NEW_ID)), AS_TYPE_DEF => TREE_VOID, AS_EXP => ACTUAL_EXP);¶¶      when DN_IN_OUT_ID =>¶        SUBTYPE_NODE :=
 D (SM_OBJ_TYPE,¬FORMAL_ID);¶        SUBSTITUTE (SUBTYPE_NODE, NODE_HASH, H);¶        NEW_ID   :=¬MAKE_VARIABLE_ID (SM_OBJ_TYPE¬=> SUBTYPE_NODE, SM_IN
IT_EXP => ACTUAL_EXP, SM_RENAMES_OBJ => True);¶        NEW_DECL :=¬MAKE_RENAMES_OBJ_DECL (LX_SRCPOS => SRCPOS, AS_SOURCE_NAME => NEW_ID,¬AS_TYPE_MARK_

NAME => TREE_VOID, AS_NAME¬=> ACTUAL_EXP);¶¶      when CLASS_TYPE_NAME =>¶        DEFN := GET_NAME_DEFN¬(ACTUAL_EXP);¶        if DEFN /= TREE_VOID the
n¶¬SUBTYPE_NODE := D (SM_TYPE_SPEC, DEFN);¶¬INSERT_NODE_HASH (NODE_HASH, GET_BASE_TYPE (SUBTYPE_NODE), GET_BASE_STRUCT (D (SM_TYPE_SPEC, FORMAL_ID)));
¶¬SUBSTITUTE (SUBTYPE_NODE, NODE_HASH, H);¶¬¬¬¬¬-- $$$$ CHECK THAT DIMENSION,¬INDEX TYPES AND COMPONENT TYPE OK¶        else¶¬SUBTYPE_NODE := TREE_VOID
;¶        end if;¶¶        NEW_ID   :=¬MAKE_SUBTYPE_ID (SM_TYPE_SPEC¬=> SUBTYPE_NODE);¶        NEW_DECL :=¬MAKE_SUBTYPE_DECL (LX_SRCPOS => SRCPOS,¬AS_
SOURCE_NAME => NEW_ID, AS_SUBTYPE_INDICATION => ACTUAL_EXP);¶¶      when DN_PROCEDURE_ID =>¶        HEADER := D¬(SM_SPEC,¬FORMAL_ID);¶        SUBSTITU
TE (HEADER, NODE_HASH, H);¶        NEW_ID := MAKE_PROCEDURE_ID (SM_SPEC => HEADER, SM_UNIT_DESC => TREE_VOID);¶        D¬(SM_FIRST, NEW_ID, NEW_ID);¶
        NEW_DECL :=¬MAKE_SUBPROG_ENTRY_DECL (LX_SRCPOS => SRCPOS, AS_SOURCE_NAME => NEW_ID, AS_HEADER => HEADER, AS_UNIT_KIND => MAKE_RENAMES_UNIT (LX_
SRCPOS => SRCPOS, AS_NAME =>¬ACTUAL_EXP));¶¶      when DN_FUNCTION_ID | DN_OPERATOR_ID =>¶        HEADER := D¬(SM_SPEC,¬FORMAL_ID);¶        SUBSTITUTE
 (HEADER, NODE_HASH, H);¶        NEW_ID := MAKE_FUNCTION_ID (SM_SPEC => HEADER, SM_UNIT_DESC => TREE_VOID);¶        D¬(SM_FIRST, NEW_ID, NEW_ID);¶
    NEW_DECL :=¬MAKE_SUBPROG_ENTRY_DECL (LX_SRCPOS => SRCPOS, AS_SOURCE_NAME => NEW_ID, AS_HEADER => HEADER, AS_UNIT_KIND => MAKE_RENAMES_UNIT (LX_SRC
POS => SRCPOS, AS_NAME =>¬ACTUAL_EXP));¶¶      when others =>¶        Put_Line ("!! BAD GENERIC ACTUAL ID");¶        raise Program_Error;¶    end case
;¶¶    D (LX_SRCPOS, NEW_ID, SRCPOS);¶    D (LX_SYMREP, NEW_ID, SYMREP);¶    NEW_DEF := MAKE_DEF_FOR_ID (NEW_ID,¬H);¶    if NEW_ID.TY in¬CLASS_SUBPROG
_NAME then¶      MAKE_DEF_VISIBLE (NEW_DEF, HEADER);¶    else¶      MAKE_DEF_VISIBLE (NEW_DEF);¶    end if;¶    INSERT_NODE_HASH (NODE_HASH, NEW_ID, F
ORMAL_ID);¶    NEW_DECL_LIST := APPEND (NEW_DECL_LIST, NEW_DECL);¶  end CONSTRUCT_INSTANCE_DECL;¶      --|-------------------------------------------
---------------------------------------------¶      --|¶  procedure FIX_DECLS_AND_SUBSTITUTE (DECL_S : TREE; NODE_HASH : in out NODE_HASH_TYPE; H¬:
H_TYPE)¬is¶    DECL_LIST : SEQ_TYPE := LIST (DECL_S);¶    DECL¬      : TREE;¶    TYPE_ID    : TREE;¶    TYPE_SPEC : TREE;¶    BASE_TYPE : TREE;¶  begin¶
  while¬not IS_EMPTY (DECL_LIST) loop¶      POP¬(DECL_LIST, DECL);¶      SUBSTITUTE (DECL, NODE_HASH, H);¶      if DECL.TY = DN_TYPE_DECL then¶
 TYPE_ID   := D (AS_SOURCE_NAME,¬DECL);¶        TYPE_SPEC := D (SM_TYPE_SPEC, TYPE_ID);¶        if TYPE_SPEC.TY = DN_ENUMERATION and then¬D (SM_RANGE
, TYPE_SPEC) = TREE_VOID then¶¬BASE_TYPE¬:= GET_BASE_TYPE (TYPE_SPEC);¶¬if D (SM_RANGE, BASE_TYPE) = TREE_VOID and then D¬(SM_DERIVED, BASE_TYPE) /= T
REE_VOID and then D (SM_RANGE, D (SM_DERIVED, BASE_TYPE)) /= TREE_VOID then¶¬  declare¶¬    THE_RANGE : TREE      := D¬(SM_RANGE, D (SM_DERIVED, BASE_T
YPE));¶¬    ENUM_S    : TREE      := D¬(SM_LITERAL_S, D (SM_DERIVED,¬BASE_TYPE));¶¬    ENUM_LIST : SEQ_TYPE := LIST (ENUM_S);¶¬    ENUM¬     : TREE;¶¬
   NEW_LIST  : SEQ_TYPE := (TREE_NIL, TREE_NIL);¶¬  begin¶¬    while¬not IS_EMPTY (ENUM_LIST) loop¶¬      POP¬(ENUM_LIST, ENUM);¶¬      NEWSNAM.REPLACE
```

```
     _SOURCE_NAME (ENUM, NODE_HASH, H);¶¬      NEW_LIST := INSERT (NEW_LIST, ENUM);¶¬    end loop;¶¬    SUBSTITUTE (ENUM_S, NODE_HASH, H);¶¬    SUBSTITUTE
     (THE_RANGE, NODE_HASH, H);¶¬    D (SM_LITERAL_S, BASE_TYPE, ENUM_S);¶¬    D (SM_RANGE, BASE_TYPE, THE_RANGE);¶¬    DI (CD_IMPL_SIZE, BASE_TYPE, DI (CD
     _IMPL_SIZE, D (SM_DERIVED, BASE_TYPE)));¶¬  end;¶¬end if;¶¬if TYPE_SPEC /= BASE_TYPE then¶¬  D (SM_RANGE, TYPE_SPEC, D (SM_RANGE, BASE_TYPE));¶¬  D (S
     M_LITERAL_S, TYPE_SPEC,¬D (SM_LITERAL_S, BASE_TYPE));¶¬  DI (CD_IMPL_SIZE,¬TYPE_SPEC, DI (CD_IMPL_SIZE, BASE_TYPE));¶¬end if;¶        end if;¶      en
     d¬if;¶    end loop;¶  end FIX_DECLS_AND_SUBSTITUTE;¶¶    --|-----------------------------------------------------------------------------------------------
     ----¶end INSTANT;¶
```